Maxey0 Context Plane — MCP server reference

A remote, read-only, unauthenticated MCP server on Cloudflare Workers.

```
https://maxey0-context-plane-mcp.maxey0.workers.dev
```

Endpoints

Path	Method	Serves
`/mcp`	POST	MCP, every domain
`/mcp/<domain id>` (11, e.g. `/mcp/agentic-systems`, `/mcp/business-finance`)	POST	MCP, one domain each: the same tools over a smaller map
`/app`	GET	The Maxey0 Context Plane Graph web app
`/graph.json`	GET	The whole map as JSON for browser apps (any origin may read it), with every node's field location; `?index=1` adds the TF-IDF index (a vector for every node on all six levels)
`/`	GET	Server info, endpoints, counts and the map's digest

A domain endpoint answers only for its own domain: looking up a node from another domain returns a tool error.

Protocol

- **MCP 2026-07-28**, stateless: requests carry `_meta["io.modelcontextprotocol/protocolVersion"]` and the `MCP-Protocol-Version`, `Mcp-Method` and `Mcp-Name` headers; `server/discover` describes the server. List results are cacheable (`ttlMs` one hour, `cacheScope: "public"`).
- **Earlier clients** (2025-11-25, 2025-06-18, 2025-03-26) use the `initialize` handshake and work the same way.
- **MCP Apps** (extension `io.modelcontextprotocol/ui`, spec 2026-01-26): `open_context_plane_graph` declares the app resource `ui://maxey0-context-plane/graph.html`.
- Requests that carry an `Origin` header must come from an allowed origin (`https://claude.ai`, `https://claude.com`); server-to-server clients send none.

Tools

All tools are read-only (`readOnlyHint: true`, `destructiveHint: false`, `idempotentHint: true`).

Tool	Arguments	Returns
`list_domains`	—	The five domains with topic counts
`list_topics`	`query?`, `domain?`	Topic ids, labels and domains, optionally filtered
`get_topic`	`topic_id`	Domain, priority, concepts with skill counts, related concepts in other topics, listed MCP servers
`get_harness`	`topic_id`	The complete topic: every concept, skill, agent, server and link (about 49 KB)

Tool	Arguments	Returns
`get_concept`	`concept_id`	The concept and all its skills with their agents
`get_skill`	`skill_id`	The skill, its path, its agents by role and each agent's servers with score, basis and source
`get_agent`	`agent_key`	One MaxeyN agent: the skills it serves in each role and the servers it holds
`get_mcp_server`	`mcp_id`	One server: description, source, the topics that list it and the agents that hold it
`search`	`query`, `limit?`, `domain?`, `use_priority?`	Matching nodes with their paths and scores
`nearest`	`node_id` or `text`, `level?`, `limit?`	Nearest nodes in the shared six-level embedding field, with scores and field locations

Tool	Arguments	Returns
route_task	task , limit? , domain? , use_priority? , mode? (flat · lens), domains?	matched , and candidate paths domain → topic → concept → skill → agents → servers. With mode: "lens" : the observer chain lens (per level: selected nodes, weights, each branch's signed contribution, suppressed candidates) and remaining_task
children	level , parent_id?	The children of a node at the next level
trace	node_id	The chain from the domain down to that node
open_context_plane_graph	task? , topic_id?	Opens the graph app (MCP App). Also returns app_url , and the best route when a task is given
validate_topic_pack	pack	ok , problems , warnings , counts and the nearest existing topics for a generated topic pack. Writes nothing.

domain is one of the 11 domain ids: agentic-systems , data-science , math-physical-sciences , engineering , life-health-mind , earth-environment-place , business-finance , society-governance , languages-communication , area-identity-studies , humanities-arts . limit is 1–50.

Observer-lens routing (mode: "lens")

The observer starts above the topic plane, or at the domains given in `domains` (superposed when more than one). It selects up to three topics, then reads the concept plane from those topics and the skill plane from the chosen concepts. A candidate x seen from a selected node p scores $\varphi = \lambda \cdot a + (1-\lambda) \cdot s$, where $a = \cos(x, p)$ and s is the cosine between what x adds beyond p and what is left of the task (it can be negative). Several selected nodes add their weighted φ ($\psi = \Sigma\, w_i \varphi_i$, $\Sigma\, w_i^2 = 1$); candidates are ranked by ψ and selected only when $\psi > 0$. λ is 0 at the topic level, 0.5 for concepts and 0.3 for skills. Agents and servers come from the ontology (roles, gates, checker inheritance), not from the lens.

Reading results

- **`matched: false`** from `route_task` means no skill shares enough words with the task. It is an answer, not an error.
- **Server links** carry a `basis`: `derived:tfidf-cosine` (with its score), `inherited:checker-of-primary` (with `via`: the skill and the primary it came from), or `declared`.
- **`use_priority`** (default `true`) breaks exact score ties by a topic's declared priority. It never changes a score.
- Unknown ids and out-of-domain lookups return `isError: true` with a message.

Prompt

`topic-graph-generator` — arguments `topic` (required), `concept`, `task`. Returns the request for generating one topic pack: the format, the rules, the existing topics, and the instruction to validate with `validate_topic_pack`.

Resources

URI	
`ctxplane://snapshot`	Count

URI	
`ctxplane://ontology`	The ru from
`ctxplane://level/{domain|topic|concept|skill|agent|mcp}`	Every and la
`ctxplane://topic/{topic_id}` (template)	One to
`ui://maxey0-context-plane/graph.html`	The g `text` app

The MCP App

`open_context_plane_graph` carries `_meta.ui.resourceUri = "ui://maxey0-context-plane/graph.html"`. Reading that resource returns the app's HTML with `_meta.ui.csp.connectDomains` set to the server it came from, because the app loads the map from that server's `/graph.json`. It loads nothing else.

Inside the host, the app:

1. sends `ui/initialize` (`appInfo`, `appCapabilities.availableDisplayModes: ["inline", "fullscreen"]`, `protocolVersion: "2026-01-26"`), then `ui/notifications/initialized`;
2. asks for a height of 900 px with `ui/notifications/size-changed`;
3. applies `ui/notifications/tool-input`: a `task` is routed and highlighted, a `topic_id` is selected;
4. sends `ui/request-display-mode` from its **Full screen** button, and `ui/message` from **Ask Claude to generate it**.

In a host without MCP Apps, the tool's text result gives `app_url`, the same app on the web.

Size of what is returned

Call	About
Tool definitions and instructions	7 KB
`get_topic`	3 KB
`get_skill`	5 KB
`route_task` (3 paths)	10 KB
`get_harness`	49 KB

The full map (about 1.9 MB) is held by the server and never sent through MCP.

Errors

Situation	Result
Unknown tool, bad arguments	JSON-RPC `-32602`
Unknown method	`-32601` (HTTP 404 on 2026-07-28)
Missing or mismatched MCP headers	`-32020` / `-32022`
Unknown id, wrong domain, invalid topic	Tool result with `isError: true`
Resource not found	`-32602` (2026-07-28) / `-32002` (earlier)
Disallowed `Origin`	HTTP 403